Assignment-Circular Singly linked list

Name: Arjun Mankuskar

PRN:202401110038

Branch: CSE (AIML)

Batch: A2

CODE-

```
#include <iostream>
#include <string>
#include <limits>

using namespace std;

struct Node {
    string data;
    Node* next;
};

void insertAtBegin(Node** head_ref, Node** last_ref, string song_title);
void insertAtLast(Node** head_ref, Node** last_ref, string song_title);
void deleteBegin(Node** head_ref, Node** last_ref);
void deleteLast(Node** head_ref, Node** last_ref);
void reverseList(Node** head_ref, Node** last_ref);
void searchSong(Node* head, string song_title);
void displayList(Node* head);
void freeList(Node** head_ref, Node** last_ref);

void showMenu() {
    cout << "\n========== Music Playlist Menu ==========\n";
    cout << " (Note: Song titles cannot contain spaces)\n";
    cout << "1. Insert Song at Begin\n";
    cout << "2. Insert Song at Last\n";
    cout << "3. Delete Song from Begin\n";
    cout << "4. Delete Song from Last\n";
    cout << "5. Reverse Playlist\n";
```

```
    cout << "6. Search for a Song\n";
    cout << "7. Display Playlist\n";
    cout << "8. Exit\n";
    cout << "=======================================\n";
    cout << "Enter your choice: ";
}


int main() {
    Node* head = nullptr;
    Node* last = nullptr;
    int choice;
    string song_title;

    do {
        showMenu();
        cin >> choice;

        if (cin.fail()) {
            cout << "\nInvalid input. Program is now in a failed state." << endl;
            continue;
        }

        switch (choice) {
            case 1:
                cout << "Enter song title (one word): ";
                cin >> song_title;
                insertAtBegin(&head, &last, song_title);
                break;

            case 2:
```

```
    cout << "Enter song title (one word): ";
    cin >> song_title;
    insertAtLast(&head, &last, song_title);
    break;

case 3:
    deleteBegin(&head, &last);
    break;

case 4:
    deleteLast(&head, &last);
    break;

case 5:
    reverseList(&head, &last);
    break;

case 6:
    cout << "Enter song title to search (one word): ";
    cin >> song_title;
    searchSong(head, song_title);
    break;

case 7:
    displayList(head);
    break;

case 8:
    cout << "Exiting the program. Goodbye!" << endl;
    break;
```

```
            default:
                cout << "Invalid choice. Please try again." << endl;
                break;
        }
    } while (choice != 8);

    freeList(&head, &last);

    return 0;
}



void insertAtBegin(Node** head_ref, Node** last_ref, string song_title) {
    Node* new_song = new Node{song_title, nullptr};
    if (*head_ref == nullptr) { *head_ref = new_song; *last_ref = new_song; new_song->next =
*head_ref; }
    else { new_song->next = *head_ref; (*last_ref)->next = new_song; *head_ref = new_song; }
    cout << "-> Added '" << song_title << "' to the beginning." << endl;
}

void insertAtLast(Node** head_ref, Node** last_ref, string song_title) {
    if (*head_ref == nullptr) { insertAtBegin(head_ref, last_ref, song_title); return; }
    Node* new_song = new Node{song_title, nullptr};
    (*last_ref)->next = new_song; new_song->next = *head_ref; *last_ref = new_song;
    cout << "-> Added '" << song_title << "' to the end." << endl;
}

void deleteBegin(Node** head_ref, Node** last_ref) {
    if (*head_ref == nullptr) { cout << "-> Playlist is empty. Nothing to delete." << endl; return; }
    Node* temp = *head_ref; string deleted_song = temp->data;
    if (*head_ref == *last_ref) { *head_ref = nullptr; *last_ref = nullptr; }
```

```
    else { *head_ref = (*head_ref)->next; (*last_ref)->next = *head_ref; }
    delete temp;
    cout << "-> Deleted '" << deleted_song << "' from the beginning." << endl;
}

void deleteLast(Node** head_ref, Node** last_ref) {
    if (*head_ref == nullptr) { cout << "-> Playlist is empty. Nothing to delete." << endl; return; }
    Node* temp = *last_ref; string deleted_song = temp->data;
    if (*head_ref == *last_ref) { *head_ref = nullptr; *last_ref = nullptr; }
    else { Node* current = *head_ref; while (current->next != *last_ref) { current = current->next; }
current->next = *head_ref; *last_ref = current; }
    delete temp;
    cout << "-> Deleted '" << deleted_song << "' from the end." << endl;
}

void reverseList(Node** head_ref, Node** last_ref) {
    if (*head_ref == nullptr || *head_ref == *last_ref) { cout << "-> Playlist has one or no songs. No
reversal needed." << endl; return; }
    Node* previous = *last_ref; Node* current = *head_ref; Node* next_node;
    do { next_node = current->next; current->next = previous; previous = current; current =
next_node; } while (current != *head_ref);
    Node* temp = *head_ref; *head_ref = *last_ref; *last_ref = temp;
    cout << "-> Playlist has been reversed." << endl;
}

void searchSong(Node* head, string song_title) {
    if (head == nullptr) { cout << "-> Playlist is empty." << endl; return; }
    Node* current = head; int position = 1; bool found = false;
    do { if (current->data == song_title) { cout << "-> Found '" << song_title << "' at position " <<
position << "." << endl; found = true; break; } current = current->next; position++; } while (current !=
head);
    if (!found) { cout << "-> Song '" << song_title << "' was not found." << endl; }
}
```

```
void displayList(Node* head) {
    if (head == nullptr) { cout << "\n-> Playlist is currently empty." << endl; return; }
    cout << "\n--- Current Playlist ---" << endl; cout << "Head -> ";
    Node* current = head;
    do { cout << current->data << " -> "; current = current->next; } while (current != head);
    cout << "(Back to Head: " << head->data << ")" << endl; cout << "------------------------" << endl;
}

void freeList(Node** head_ref, Node** last_ref) {
    if (*head_ref == nullptr) return;
    (*last_ref)->next = nullptr;
    Node* current = *head_ref; Node* next_node;
    while (current != nullptr) { next_node = current->next; delete current; current = next_node; }
    *head_ref = nullptr; *last_ref = nullptr;
    cout << "\nPlaylist memory has been freed." << endl;
}
```

OUTPUT-

```
========== Music Playlist Menu ==========
 (Note: Song titles cannot contain spaces)
1. Insert Song at Begin
2. Insert Song at Last
3. Delete Song from Begin
4. Delete Song from Last
5. Reverse Playlist
6. Search for a Song
7. Display Playlist
8. Exit
========================================
Enter your choice: 1
```

Enter song title (one word): Sabr

-> Added 'Sabr' to the beginning.

========== Music Playlist Menu ==========

(Note: Song titles cannot contain spaces)

1. Insert Song at Begin

2. Insert Song at Last

3. Delete Song from Begin

4. Delete Song from Last

5. Reverse Playlist

6. Search for a Song

7. Display Playlist

8. Exit

==

Enter your choice: 1

Enter song title (one word): Goodbye

-> Added 'Goodbye' to the beginning.

========== Music Playlist Menu ==========

(Note: Song titles cannot contain spaces)

1. Insert Song at Begin

2. Insert Song at Last

3. Delete Song from Begin

4. Delete Song from Last

5. Reverse Playlist

6. Search for a Song

7. Display Playlist

8. Exit

==

Enter your choice: 2

Enter song title (one word): Shorgul

-> Added 'Shorgul' to the end.

========== Music Playlist Menu ==========

(Note: Song titles cannot contain spaces)

1. Insert Song at Begin

2. Insert Song at Last

3. Delete Song from Begin

4. Delete Song from Last

5. Reverse Playlist

6. Search for a Song

7. Display Playlist

8. Exit

==

Enter your choice: 2

Enter song title (one word): Umiya

-> Added 'Umiya' to the end.

========== Music Playlist Menu ==========

(Note: Song titles cannot contain spaces)

1. Insert Song at Begin

2. Insert Song at Last

3. Delete Song from Begin

4. Delete Song from Last

5. Reverse Playlist

6. Search for a Song

7. Display Playlist

8. Exit

==

Enter your choice: 3

-> Deleted 'Goodbye' from the beginning.

========== Music Playlist Menu ==========

(Note: Song titles cannot contain spaces)

1. Insert Song at Begin

2. Insert Song at Last

3. Delete Song from Begin

4. Delete Song from Last

5. Reverse Playlist

6. Search for a Song

7. Display Playlist

8. Exit

==

Enter your choice: 5

-> Playlist has been reversed.

========== Music Playlist Menu ==========

(Note: Song titles cannot contain spaces)

1. Insert Song at Begin

2. Insert Song at Last

3. Delete Song from Begin

4. Delete Song from Last

5. Reverse Playlist

6. Search for a Song

7. Display Playlist

8. Exit

==

Enter your choice: 6

Enter song title to search (one word): Shorgul

-> Found 'Shorgul' at position 2.

========== Music Playlist Menu ==========

(Note: Song titles cannot contain spaces)

1. Insert Song at Begin

2. Insert Song at Last

3. Delete Song from Begin

4. Delete Song from Last

5. Reverse Playlist

6. Search for a Song

7. Display Playlist

8. Exit

==========

Enter your choice: 7

--- Current Playlist ---

Head -> Umiya -> Shorgul -> Sabr -> (Back to Head: Umiya)

========== Music Playlist Menu ==========

(Note: Song titles cannot contain spaces)

1. Insert Song at Begin

2. Insert Song at Last

3. Delete Song from Begin

4. Delete Song from Last

5. Reverse Playlist

6. Search for a Song

7. Display Playlist

8. Exit

==========

Enter your choice: 4

-> Deleted 'Sabr' from the end.

========== Music Playlist Menu ==========

(Note: Song titles cannot contain spaces)

1. Insert Song at Begin

2. Insert Song at Last

3. Delete Song from Begin

4. Delete Song from Last

5. Reverse Playlist

6. Search for a Song

7. Display Playlist

8. Exit

==========

Enter your choice: 7

--- Current Playlist ---

Head -> Umiya -> Shorgul -> (Back to Head: Umiya)

========== Music Playlist Menu ==========

(Note: Song titles cannot contain spaces)

1. Insert Song at Begin

2. Insert Song at Last

3. Delete Song from Begin

4. Delete Song from Last

5. Reverse Playlist

6. Search for a Song

7. Display Playlist

8. Exit

==========

Enter your choice: 8

Exiting the program. Goodbye!

Playlist memory has been freed.
